Git Merge Conflict – Hands-on Exercise

Git Merge Conflict — Theory Explanation

A **merge conflict** happens when Git is unable to automatically combine changes from two branches.

1. What is a Merge?

Suppose a project has two branches:

main/master

feature-login

You create the feature-login branch to work on a new feature. After completing the work, you want to bring those changes back into the main or master branch.

This process is called a **merge**.

git merge feature-login

Usually, Git can combine the changes automatically.

2. Why Does a Merge Conflict Happen?

A conflict usually happens when **two branches modify the same part of the same file differently**.

For example, initially both branches have:

Hello Everyone

Then the branches change separately.

On feature-login

Hello Developer

On main or master

Hello Everyone!

The situation becomes:

Original

Hello Everyone

|

| |

↓ ↓

main/master feature-login

| |

↓ ↓

Hello Everyone! Hello Developer

Now when you try:

git merge feature-login

Git sees two different changes to the **same line**.

Git does not know which version is correct:

Hello Everyone!

or:

Hello Developer

Therefore, Git stops the merge and asks you to decide.

This situation is called a **merge conflict**.

3. Conflict Markers

Git places special markers inside the conflicting file.

For example:

<<<<<<< HEAD

Hello Everyone!

=======

Hello Developer

>>>>>>> feature-login

These markers help you understand the conflict.

<<<<<<< HEAD

This indicates the beginning of the version from your **current branch**.

In this example, you are currently on main or master.

<<<<<<< HEAD

Hello Everyone!

So Hello Everyone! belongs to your current branch.

=======

This separates the two versions.

=======

Everything above it belongs to the current branch, and everything below it belongs to the branch being merged.

>>>>>>> feature-login

This indicates the end of the incoming changes.

Hello Developer

>>>>>>> feature-login

This version comes from the feature-login branch.

4. How Do You Resolve the Conflict?

Git does not automatically choose a version because only the developer knows which change should be kept.

You have three choices.

Choice 1: Keep the Current Branch Version

Hello Everyone!

Choice 2: Keep the Incoming Branch Version

Hello Developer

Choice 3: Combine or Create Your Own Version

Hello Everyone and Developers!

You manually edit the file and remove all conflict markers.

5. Tell Git That the Conflict Is Resolved

After editing the file, Git still needs confirmation that you have resolved the conflict.

You use:

git add message.txt

In a normal situation, git add stages a file.

During a merge conflict, it also tells Git:

“I have fixed this file, and this is the final version I want to use.”

6. Complete the Merge

After resolving and staging the file, run:

git commit -m "Resolve merge conflict"

This creates the merge commit and completes the merge process.

Complete Theory Flow

Two branches are created

↓

Both branches modify the same line differently

↓

You try to merge the branches

↓

Git cannot decide which change to keep

↓

Git creates a merge conflict

↓

Git adds conflict markers to the file

↓

Developer manually chooses the final content

↓

Remove conflict markers

↓

git add

↓

git commit

↓

Merge Conflict Resolved ✅

In one sentence:

A **Git merge conflict occurs when Git cannot automatically combine changes from different branches and requires the developer to manually decide the final version of the conflicting code or file.**

Exercise: Create and Resolve a Merge Conflict

Step 1: Create a New Repository

```
mkdir git-conflict-demo
cd git-conflict-demo
git init
```

Create message.txt with the text Hello Everyone:

```
echo "Hello Everyone" > message.txt
```

Check the file:

```
cat message.txt
```

Add and commit it:

```
git add message.txt
git commit -m "Initial message"
```

Step 2: Create a Feature Branch

```
git switch -c feature-login
```

Change message.txt:

```
echo "Hello Developer" > message.txt
```

Check the file:

```
cat message.txt
```

Commit the change:

```
git add message.txt
git commit -m "Update message for developers"
```

Step 3: Switch Back to Main/Master

Depending on your Git configuration, your default branch may be called main or master. Use the branch name shown by git branch.

```
git switch main
```

If your branch is named master, use:

```
git switch master
```

Change the same line:

```
echo "Hello Everyone!" > message.txt
```

Check the file:

```
cat message.txt
```

Commit the change:

```
git add message.txt
git commit -m "Update message for everyone"
```

Step 4: Merge the Feature Branch

```
git merge feature-login
```

Git should report a merge conflict. Check the status:

```
git status
```

Step 5: View the Conflict

Display the file:

```
cat message.txt
```

You may see:

```
<<<<<<< HEAD
Hello Everyone!
=======
Hello Developer
>>>>>>> feature-login
```

Why Did the Conflict Happen?

No problem — think of it like two people editing the same sentence.

Initially, both branches had:

```
Hello Everyone
```

Then you created the feature-login branch.

On feature-login, you changed it to:

```
Hello Developer
```

At the same time, on main/master, you changed that same line to:

```
Hello Everyone!
```

So now:

```
            Original
          Hello Everyone
                |
          ----------------
          |              |
    main/master      feature-login
          |              |
          ↓              ↓
Hello Everyone!    Hello Developer
```

Now you run:

```
git merge feature-login
```

Git is confused because it sees two different answers for the same line:

Should I keep Hello Everyone!?

or

Should I keep Hello Developer?

Git cannot decide which version you want, so it stops and says: You decide.

This is what it means when Git says that the same part of the file was changed differently.

```
Original:        Hello Everyone

main/master:     Hello Everyone!

feature-login:   Hello Developer
```

Because both branches changed the same line differently, Git creates a conflict and waits for you to choose the final version.

Step 6: Resolve the Conflict

Suppose you decide to keep:

```
Hello Developer
```

Replace the entire content of the file:

```
echo "Hello Developer" > message.txt
```

Check the file:

```
cat message.txt
```

Output:

```
Hello Developer
```

Step 7: Complete the Merge

Tell Git that the conflict has been resolved:

```
git add message.txt
```

Then complete the merge:

```
git commit -m "Resolve merge conflict"
```

Step 8: Verify

```
git log --oneline --graph
```

Complete Flow

```
Create file using echo
        ↓
Create feature branch
        ↓
Change same file differently
        ↓
Switch to main/master
        ↓
Change same line differently
        ↓
Merge feature branch
        ↓
Merge Conflict
        ↓
Resolve using echo
        ↓
git add message.txt
        ↓
git commit
        ↓
Conflict Resolved
```

Practice Exercise 1: Keep the Current Branch Version

Task: Create a conflict and resolve it by keeping the version from the current branch.

Question

Suppose message.txt contains:

```
Welcome to Git
```

On feature branch, change it to:

```
Welcome Developers
```

On main/master, change it to:

```
Welcome Students
```

Merge the feature branch. A conflict occurs. Resolve the conflict by keeping:

```
Welcome Students
```

Answer

```
echo "Welcome Students" > message.txt
git add message.txt
git commit -m "Resolve conflict and keep student message"
```

Final content of message.txt:

```
Welcome Students
```

Practice Exercise 2: Combine Both Changes

Task: Create a conflict and combine both versions into one final message.

Question

Suppose the original file contains:

```
Learn Git
```

On feature branch, change it to:

```
Learn Git Branching
```

On main/master, change it to:

```
Learn Git Commands
```

Merge the feature branch. Resolve the conflict by combining both messages into:

```
Learn Git Commands and Branching
```

Answer

```
echo "Learn Git Commands and Branching" > message.txt
git add message.txt
git commit -m "Resolve conflict by combining changes"
```

Final content of message.txt:

```
Learn Git Commands and Branching
```

Important Commands to Remember

`git status`	Check the current Git status
`cat message.txt`	View the file content
`git add message.txt`	Mark the resolved file as ready
`git commit -m "message"`	Complete the merge
`git merge --abort`	Cancel the merge if needed